INTERNATIONAL COMPUTER SCIENCE COMPETITION

Edition of 2026 — Qualification Round Solution Document

Participant Account ID: ICSC-2026-94812 **Submission Date:** June 9, 2026

Track: Computer Science (Qualification) **Total Points Target:** 25/25

Problem A: A History of Computing

Based on the timeline graph tracking the exponential expansion of transistor densities on commercial microprocessors against notable milestones, the designated historic technology events are identified below:

Event No.	Year	Identified Milestone / Historic Technology Breakthrough
1	1936	Alan Turing publishes "On Computable Numbers," introducing the universal Turing Machine concept.
2	1947	Invention of the point-contact transistor by John Bardeen, Walter Brattain, and William Shockley at Bell Labs.
3	1958	Development of the first working Integrated Circuit (IC) by Jack Kilby at Texas Instruments.
4	1969	Deployment of the ARPANET (first node-to-node connection) and the initial creation of the UNIX Operating System.
5	1981	Launch of the IBM Personal Computer (IBM PC Model 5150), establishing the generic architecture for modern PCs.
6	1991	The World Wide Web (WWW) is opened to the general public by Tim Berners-Lee at CERN.
7	2000	Release of the first commercial 1 GHz microprocessors (AMD Athlon and Intel Pentium III).
8	2007	Introduction of the Apple iPhone, catalyzing the global mobile and smartphone computing era.
9	2012	AlexNet wins the ImageNet competition, initiating the modern deep learning and GPU-accelerated AI boom.
10	2022	Release of ChatGPT by OpenAI, igniting the mainstream generative AI and large language model (LLM) revolution.

Problem B: Pixel Art

Below is the complete clean implementation of the `generate_shape` function. The design leverages structural mathematics to populate the matrix efficiently without relying on nested conditional overrides.

Python 3.9 Implementation

```
def generate_shape(n: int, shape: str) -> list:
    grid = [[0] * n for _ in range(n)]

    if shape == "checkerboard":
        for i in range(n):
            for j in range(n):
                # Flips values alternate across index sums; top-left (0,0) evaluates
to 0
                grid[i][j] = (i + j) % 2

    elif shape == "diamond":
        center = n // 2
        for i in range(n):
            for j in range(n):
                # Manhattan distance logic ensures exact diamond contour formatting
                if abs(i - center) + abs(j - center) <= center:
                    grid[i][j] = 1

    return grid

# Standard execution formatting block
if __name__ == "__main__":
    import sys
    # Example execution configuration hook
    # lines = sys.stdin.read().split()
    pass
```

Problem C: Moore's Law

(a) Mathematical Prediction for 2026

The standard model provided is given by the formula:

$$T(t) = T_0 \times 2^{t/2}$$

Given baseline coefficients from the 1971 Intel 4004 architecture release:

- Reference Count (T_0) = 2,300 transistors
- Elapsed Temporal Offset (t) = 2026 − 1971 = 55 years

Substituting the parameters into the analytical function:

$$T(55) = 2300 \times 2^{55/2} = 2300 \times 2^{27.5}$$

Calculating the exponential scalar components:

$$2^{27.5} = 2^{27} \times \sqrt{2} \approx 134,217,728 \times 1.41421356 \approx 189,812,531.02$$

$$T(55) = 2300 \times 189,812,531.02 \approx 436,568,821,346$$

Thus, the theoretical transistor count predicted for a 2026 microprocessor is approximately **436.57 Billion transistors**.

(b) Empirical Comparison with Modern Hardware

The base model of the commercial 2026 Apple M5 processor incorporates an actual transistor infrastructure density count of **28 Billion transistors** (fabricated on TSMC's enhanced 3nm node architecture). Comparing the two values:

$$Ratio = Theoretical / Empirical = 436.57B / 28B \approx 15.59x$$

Evaluation: Over a prolonged 55-year timeline, Moore's raw structural formulation overestimates the physical implementation by roughly a factor of 15.6. This manifests because physical, electrical, and thermal leakage ceilings slowed historical doubling trends from 24 months to roughly 30–36 months during the sub-10nm fabrication scaling phases.

(c) Recalculation of the True Doubling Period

To evaluate the actual compounding exponential coefficient, we isolate the variable d (doubling period) in the modified expression:

$$T_{actual} = T_0 \times 2^{t/d}$$

$$28,000,000,000 = 2,300 \times 2^{55/d}$$

$$2^{55/d} = 28,000,000,000 / 2,300 \approx 12,173,913.04$$

Applying natural log transforms to resolve dimensions:

$$(55/d) \times \ln(2) = \ln(12,173,913.04)$$

$$(55/d) \times 0.693147 = 16.314777$$

$$55/d = 16.314777 / 0.693147 \approx 23.53725$$

$$d = 55 / 23.53725 \approx 2.3367$$

The authentic, empirical doubling interval that captures the evolution from the Intel 4004 to contemporary silicon is **2.34 years** (approximately 28 months).

Problem D: The Simultaneous Strike

(a) Architectural Bug Analysis

The algorithmic breakdown inside the provided game engine pseudocode stems from a structural state-check ordering flaw. By enforcing a conditional check to `break` processing immediate upon either player's local parameters hitting `hp ≤ 0` within the internal iterative evaluation loop, it breaks atomic handling principles across identical timestamps (frames).

For instance, in the log sequence provided, both players execute uniform damage vectors (20 HP) at frame 512. Since the logs undergo standard stable sort arrangements, Player 1's action is evaluated first. It decrements Player 2's HP value down to −5 (clamped to 0). On the immediate sequential loop pass, line 3 registers `player2_hp ≤ 0` and terminates execution via a `break` rule. Consequently, Player 2's corresponding frame damage payload is dumped completely. This introduces an artificial bias toward whoever happens to occupy the early index position post-sorting, failing to honor the simultaneous strike rules which specify both events must evaluate prior to terminal verification sweeps.

(b) Corrected Functional Implementation

The code below groups and processes all concurrent frame updates fully before performing any terminal health validation evaluations.

```
def processGame(events: list, H: int) -> list:
    # Handle empty array edge conditions
    if not events:
        return [H, H]

    # Step 1: Stable sort by frame index (element index 1)
    sorted_events = sorted(events, key=lambda x: x[1])

    p1_hp = H
    p2_hp = H

    i = 0
    num_events = len(sorted_events)

    while i < num_events:
        # Check current termination vectors before starting a new frame block
        if p1_hp <= 0 or p2_hp <= 0:
            break

        current_frame = sorted_events[i][1]
        frame_end_index = i

        # Accumulate all operations executing inside the same frame group
        while frame_end_index < num_events and sorted_events[frame_end_index][1] ==
current_frame:
            frame_end_index += 1

        # Apply all attack payloads occurring on this frame simultaneously
```

```
        for idx in range(i, frame_end_index):
            player, _, attack = sorted_events[idx]
            if player == 1:
                p2_hp -= attack
            elif player == 2:
                p1_hp -= attack

        # Advance iterator index past the processed frame cluster
        i = frame_end_index

    # Enforce standard boundaries clamping value limits cleanly to a minimum of 0
    return [max(0, p1_hp), max(0, p2_hp)]
```

Problem E: PageRank in the Age of AI Slop

(a) PageRank Theoretical Framework

The PageRank algorithm models a stationary probability distribution across web structures using a *Random Surfer Model*. A surfer moves through a directed graph of pages by following out-links with probability d, or teleports to a random page anywhere across the entire network with probability $1 - d$.

The formulation is fundamentally recursive because a page's systemic authority is a linear combination of the authority metrics of its incoming link sources. In practice, this is solved by framing the system as a Markov chain transition matrix and computing the principal eigenvector using **Power Iteration methods**, iterating $PR_{k+1} = M \cdot PR_k$ until the convergence delta falls below a designated threshold.

(b) Derivation of the Total AI PageRank Expression

Let the total web topology contain N pages, partitioned into human-generated pages (H) and adversarial AI units (A). The fractional size of AI pages is represented as:

$$\alpha = a / N \Rightarrow h / N = 1 - \alpha$$

The structural linking behavior specified dictates:

- Human pages distribute links uniformly across any page in N. Thus, the fraction of human outbound links directed toward the AI cluster is exactly α, and the fraction remaining within the human cluster is $1 - \alpha$.
- AI pages structurally link exclusively to target pages within the AI subset A, leaking zero link authority back to human spaces.

We sum the standard PageRank recurrence relation over all pages $p_i \in A$:

$$S_A = \sum_{p_i \in A} [\, (1-d)/N + d \cdot \sum_{p_j \in M(p_i)} (PR(p_j) / L(p_j)) \,]$$

Summing the constant factor over the AI subset (which contains a nodes) yields:

$$\sum_{p_i \in A} (1-d)/N = a \cdot (1-d)/N = \alpha(1-d)$$

Next, we break down the total incoming link authority into contributions from human nodes and AI nodes:

$$\sum_{p_i \in A} \sum_{p_j \in M(p_i)} (PR(p_j) / L(p_j)) = \alpha \cdot S_H + 1.0 \cdot S_A$$

Since the total probability space is closed, we use the constraint $S_H = 1 - S_A$ to substitute terms:

$$S_A = \alpha(1-d) + d\,[\,\alpha(1 - S_A) + S_A\,]$$

Expanding the linear terms:

$$S_A = \alpha - \alpha d + \alpha d - \alpha d\, S_A + d\, S_A$$

$$S_A = \alpha + d\, S_A(1 - \alpha)$$

Grouping all S_A factors on the left side:

$$S_A\,[\,1 - d(1 - \alpha)\,] = \alpha$$

Isolating S_A provides the final closed-form expression:

$$S_A = \alpha / [\,1 - d(1 - \alpha)\,]$$

(c) Tipping Fraction Analysis (α^*)

The systemic tipping point occurs when AI pages capture exactly half of the web's structural authority, meaning $S_A = S_H = 0.5$:

$$0.5 = \alpha^* / [\,1 - d(1 - \alpha^*)\,]$$

$$1 - d + d\alpha^* = 2\alpha^*$$

$$1 - d = \alpha^*(2 - d)$$

$$\alpha^* = (1 - d) / (2 - d)$$

Evaluating this tipping point at the standard damping factor of $d = 0.85$:

$$\alpha^* = (1 - 0.85) / (2 - 0.85) = 0.15 / 1.15 = 3 / 23 \approx 0.13043$$

Conclusion & Architectural Implications: The tipping fraction is approximately **13.04%**. This indicates that even if AI slop networks represent a clear minority of total web nodes, their specialized, self-referential linking structures form an authority sink. Because PageRank assumes that outbound links reflect genuine, non-strategic content curation, it is highly vulnerable to adversarial clusters that recycle authority internally. This dynamic forces modern commercial search engines to heavily discount pure graph metrics in favor of content quality and user interaction analysis.